

Parte III: Processamento de Linguagem Natural para Linguistas

Unidade 3 - Respostas dos Exercícios

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

Agosto de 2026

Respostas — Encontrando padrões linguísticos com o spaCy

Parte A — Conceitos

Resposta 1.

- O *Matcher* casa **sequências lineares de *Tokens*** com base em seus atributos (classe gramatical, traços morfológicos, forma, lema etc.). É adequado, por exemplo, para localizar sequências “pronome + verbo”.
- O *DependencyMatcher* casa **padrões na árvore de dependências**, isto é, relações de cabeça e dependente. É adequado para localizar, por exemplo, todos os sujeitos nominais de verbos, independentemente da ordem linear das palavras.
- O *PhraseMatcher* casa **objetos *Doc* contra objetos *Doc*** e é otimizado para listas grandes de termos fixos. É adequado, por exemplo, para localizar milhares de nomes de medicamentos ou de topônimos em um corpus.

Resposta 2.

A chave OP define a **quantificação** de um padrão de *Token*:

- ! — nega o padrão; ele deve ocorrer exatamente zero vezes.
- ? — torna o padrão opcional; pode ocorrer zero ou uma vez.
- + — exige uma ou mais ocorrências.
- * — permite zero ou mais ocorrências.

O operador ? é preferível ao + quando o elemento é **facultativo, mas não repetível**, como o determinante em um sintagma nominal do inglês: [{ 'POS': 'DET', 'OP': '?' }, { 'POS': 'NOUN' }] casa tanto *the capital* quanto *capitals*, mas não permitiria uma sequência de determinantes.

Resposta 3.

POS refere-se à etiqueta **geral** (*coarse-grained*) de classe gramatical, definida pelo conjunto universal das Dependências Universais (NOUN, VERB, ADJ, PROPN etc.). TAG refere-se à etiqueta **refinada** (*fine-grained*), específica do corpus de treinamento do modelo — no caso do inglês, o conjunto do Penn Treebank (NN, NNS, NNP, VBD etc.).

NNP é uma etiqueta do Penn Treebank e, portanto, só é válida como valor de TAG. O equivalente geral seria PROPN em POS, mas NNP é mais específico, pois distingue nome próprio singular de plural (NNPS).

Resposta 4.

Quando o valor de MORPH é uma *string*, o spaCy exige uma correspondência **exata** de todo o conjunto de traços morfológicos. Com IS_SUPERSET, o spaCy verifica se o conjunto de traços do *Token* **contém** o conjunto fornecido no padrão, ou seja, se é um superconjunto dele.

Isso é necessário porque quase nunca conhecemos de antemão a combinação completa de traços de um *Token*. Ao buscar apenas verbos no passado, por exemplo, queremos casar tanto `Tense=Past|VerbForm=Fin` quanto `Aspect=Perf|Tense=Past|VerbForm=Part`; um único traço, `Tense=Past`, basta como critério.

Resposta 5.

RIGHT_ID é o **nome do padrão atual**, usado para que padrões posteriores possam referenciá-lo. LEFT_ID é a **referência a um padrão já definido**, ao qual o padrão atual se conecta por meio do operador relacional em REL_OP.

O primeiro dicionário não possui LEFT_ID porque é a **âncora**: não há nenhum padrão anterior ao qual ele possa se ligar. A âncora é o ponto de partida da corrente.

Resposta 6.

Um objeto *Span* representa uma **fatia contígua** de um objeto *Doc*, definida por um índice inicial e um final. Correspondências de dependência, porém, ligam *Tokens* que podem estar arbitrariamente distantes na ordem linear e com material interveniente que não faz parte da correspondência (por exemplo, em *the movement in New York began*, o sujeito **movement** e o verbo **began** não são adjacentes). Como não formam uma fatia contígua, não podem ser representados por um *Span*; por isso o spaCy devolve os índices individuais dos *Tokens*.

Parte B — Programação

Resposta 7.

```
from spacy.matcher import Matcher

# Cria um novo Matcher com o vocabulário do modelo
adj_matcher = Matcher(vocab=nlp.vocab)
```

```

# Define o padrão: um adjetivo seguido de um substantivo
adj_noun = [{'POS': 'ADJ'}, {'POS': 'NOUN'}]

# Adiciona o padrão ao Matcher
adj_matcher.add('adj+noun', patterns=[adj_noun])

# Aplica o Matcher ao Doc e obtém os resultados como Spans
adj_results = adj_matcher(doc, as_spans=True)

# Imprime as 20 primeiras correspondências
for result in adj_results[:20]:
    print(result)

```

Resposta 8.

```

# Permite um ou mais adjetivos antes do substantivo
adj_noun_plus = [{'POS': 'ADJ', 'OP': '+'}, {'POS': 'NOUN'}]

# Versão sem 'greedy'
matcher_a = Matcher(vocab=nlp.vocab)
matcher_a.add('adj+noun', patterns=[adj_noun_plus])
results_a = matcher_a(doc, as_spans=True)

# Versão com 'greedy'
matcher_b = Matcher(vocab=nlp.vocab)
matcher_b.add('adj+noun', patterns=[adj_noun_plus], greedy='LONGEST')
results_b = matcher_b(doc, as_spans=True)

print('Sem greedy:', len(results_a))
print('Com greedy:', len(results_b))

```

Explicação. Sem greedy, o operador + devolve uma correspondência a cada posição em que o padrão pode ser satisfeito: para a sequência *large public space*, o spaCy devolve tanto *public space* quanto *large public space*. Com `greedy='LONGEST'`, o spaCy filtra as correspondências sobrepostas e mantém apenas a mais longa de cada grupo, reduzindo o total de resultados.

Resposta 9.

```

# Cria um Matcher para traços morfológicos
plural_matcher = Matcher(vocab=nlp.vocab)

# Define o padrão: substantivos cujos traços contêm Number=Plur
plural_noun = [{'POS': 'NOUN', 'MORPH': {'IS_SUPERSET': ['Number=Plur']}}]

# Adiciona o padrão ao Matcher
plural_matcher.add('plural_noun', patterns=[plural_noun])

```

```

# Aplica o Matcher ao Doc
plural_results = plural_matcher(doc, as_spans=True)

# Imprime as 15 primeiras correspondências e seus traços morfológicos
for result in plural_results[:15]:
    print(result, '\t', result[0].morph)

```

Resposta 10.

```

from spacy.matcher import DependencyMatcher

# Cria um novo DependencyMatcher
obj_matcher = DependencyMatcher(vocab=nlp.vocab)

# Define a corrente: âncora (verbo) e seu objeto direto
verb_dobj = [{'RIGHT_ID': 'verb', 'RIGHT_ATTRS': {'POS': 'VERB'}},
              {'LEFT_ID': 'verb', 'REL_OP': '>', 'RIGHT_ID': 'd_object',
               'RIGHT_ATTRS': {'DEP': 'dobj'}}
              ]

# Adiciona o padrão ao matcher
obj_matcher.add('verb_dobj', patterns=[verb_dobj])

# Aplica o matcher ao Doc
obj_matches = obj_matcher(doc)

# Imprime as 20 primeiras correspondências
for match in obj_matches[:20]:

    # Desempacota a tupla em nome do padrão e lista de índices
    pattern_name, matches = match[0], match[1]

    # Os índices seguem a ordem em que os padrões foram definidos
    verb, dobject = matches[0], matches[1]

    print(nlp.vocab[pattern_name].text, '\t', doc[verb], '...', doc[dobject])

```

Resposta 11.

```

# O terceiro elo parte de 'd_object', e não da âncora 'verb'
verb_dobj_det = [{'RIGHT_ID': 'verb', 'RIGHT_ATTRS': {'POS': 'VERB'}},
                  {'LEFT_ID': 'verb', 'REL_OP': '>', 'RIGHT_ID': 'd_object',
                   'RIGHT_ATTRS': {'DEP': 'dobj'}},
                  {'LEFT_ID': 'd_object', 'REL_OP': '>', 'RIGHT_ID': 'determiner',
                   'RIGHT_ATTRS': {'DEP': 'det'}}
                  ]

```

```

# Cria um matcher novo para evitar misturar com padrões anteriores
det_matcher = DependencyMatcher(vocab=nlp.vocab)
det_matcher.add('verb_dobj_det', patterns=[verb_dobj_det])

det_matches = det_matcher(doc)

for match in det_matches[:20]:

    pattern_name, matches = match[0], match[1]

    # A ordem dos índices segue a ordem de definição dos padrões
    verb, dobject, determiner = matches[0], matches[1], matches[2]

    print(nlp.vocab[pattern_name].text, '\t',
          doc[verb], '...', doc[determiner], doc[dobject])

```

Explicação. O determinante é dependente do **substantivo**, não do verbo. Por isso LEFT_ID deve ser d_object: estamos prolongando a corrente para a direita a partir do objeto direto, e não abrindo uma nova corrente a partir da âncora.

Resposta 12.

```

from wasabi import Printer

# Inicializa o Printer
match = Printer()

# Percorre as correspondências mantendo a contagem
for i, result in enumerate(adj_results, start=1):

    print(i,
          doc[result.start - 5: result.start],      # contexto à esquerda
          match.text(result, color='blue', no_print=True), # correspondência em azul
          doc[result.end: result.end + 5]           # contexto à direita
    )

```

Parte C — Reflexão

Resposta 13.

O texto de origem contém quebras de linha (`\n`) que o spaCy preserva como *Tokens*. Quando uma dessas quebras cai na janela de contexto, ela é impressa literalmente e desloca a saída para outra linha.

Uma solução é substituir os caracteres de espaçamento pelo caractere de espaço antes de imprimir:

```

for i, result in enumerate(adj_results, start=1):

```

```

# Converte as fatias em texto e normaliza os espaços em branco
left = ' '.join(doc[result.start - 5: result.start].text.split())
right = ' '.join(doc[result.end: result.end + 5].text.split())

print(i, left, match.text(result, color='blue', no_print=True), right)

```

O método `split()` sem argumentos divide a *string* em qualquer sequência de espaços em branco — incluindo `\n` e `\t` — e `join()` a reconstitui com espaços simples.

Alternativamente, poderíamos limpar o texto antes de fornecê-lo ao modelo de linguagem, ou filtrar os *Tokens* cujo atributo `is_space` seja `True`.

Resposta 14.

A construção envolve uma relação **sintática**, e não meramente linear: o sintagma nominal é o objeto da preposição, que por sua vez é dependente do verbo. Palavras podem intervir entre os elementos (*depend heavily on the government*), o que torna o *Matcher* linear frágil. Portanto, a escolha adequada é o **DependencyMatcher**.

A regra teria três elos:

```

verb_prep_np = [
    # Âncora: o verbo
    {'RIGHT_ID': 'verb', 'RIGHT_ATTRS': {'POS': 'VERB'}},

    # A preposição é dependente do verbo, com a relação 'prep'
    {'LEFT_ID': 'verb', 'REL_OP': '>', 'RIGHT_ID': 'preposition',
     'RIGHT_ATTRS': {'DEP': 'prep'}},

    # O objeto da preposição é dependente da preposição, com a relação 'pobj'
    {'LEFT_ID': 'preposition', 'REL_OP': '>', 'RIGHT_ID': 'prep_object',
     'RIGHT_ATTRS': {'DEP': 'pobj'}}
]

```

Justificativa das escolhas:

- A âncora é o verbo, porque é o elemento em torno do qual a construção se organiza.
- O segundo elo parte do verbo (`LEFT_ID: 'verb'`), pois a preposição é dependente dele.
- O terceiro elo parte da preposição (`LEFT_ID: 'preposition'`), e **não** do verbo, porque o sintagma nominal é dependente da preposição — trata-se de um prolongamento da mesma corrente.
- Se quiséssemos restringir o verbo a um lema específico, bastaria acrescentar `'LEMMA': 'depend'` ao dicionário `RIGHT_ATTRS` da âncora.